SPC · MT CAT-3

Predicted Coding Question Bank with Complete C Solutions

Course: 26CSE0105 · Structured Programming Concepts

Syllabus: Lectures 16–20 · Iteration, nested loops, jump statements, patterns and problem-solving

Prepared from the CAT-3 syllabus and the SPC preparation notes available from your previous study session. Questions are exam-style predictions for practice; they are not confirmed or guaranteed exam questions.

How to use this bank

1. Read the question and attempt the program yourself first.
2. Dry-run your loop variables before compiling.
3. Compare your code with the solution and test the sample input.
4. Practise the pattern questions by understanding the row/column relationship, not by memorising code.

Syllabus coverage and exam priorities

Topic	Practice included
for, while, do-while	Counting, reverse counting, sum and input-controlled repetition
Nested loops	Star/number/alphabet patterns and row-column logic
Infinite loops	Recognising missing/wrong updates and correcting loop conditions
break, continue, goto	Control-flow programs and loop filtering
Iteration problem-solving	Factorial, Fibonacci, digits, palindrome, prime and related problems

High-priority revision: nested-loop patterns, factorial/Fibonacci, reverse and sum of digits, palindrome, prime check, and the difference between break and continue. Also practise boundary cases such as n = 0, n = 1 and negative input.

Part A · Loop fundamentals and control flow

1. Print numbers from 1 to N (for loop)

Question: Read an integer N and print all integers from 1 through N. If N is less than 1, print nothing.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        printf("%d ", i);
    }
    return 0;
}
```

Sample: Input: 5 Output: 1 2 3 4 5

Logic: The counter starts at 1 and the loop continues while i <= n.

2. Print numbers from N down to 1 (while loop)

Question: Read N and print numbers in descending order using a while loop.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    while (n >= 1) {
        printf("%d ", n);
        n--;
    }
    return 0;
}
```

Sample: Input: 5 Output: 5 4 3 2 1

Logic: Decrement n on every iteration so the condition eventually becomes false.

3. Demonstrate do-while with at-least-once execution

Question: Read an integer and use a do-while loop to print its value once, then stop when the loop condition is false.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    do {
        printf("%d ", n);
        n--;
    } while (n > 0);

    return 0;
}
```

Sample: Input: 3 Output: 3 2 1

Logic: A do-while checks its condition after executing the body. It always runs at least once.

4. Sum of first N natural numbers

Question: Read N and calculate 1 + 2 + ... + N using iteration.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    long long sum = 0;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    printf("%lld", sum);
    return 0;
}
```

Sample: Input: 5 Output: 15

Logic: sum must start at 0. Use a wider integer type for a larger total.

5. Print even numbers up to N

Question: Read N and print all even numbers from 2 up to N.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 2; i <= n; i += 2) {
        printf("%d ", i);
    }
    return 0;
}
```

Sample: Input: 10 Output: 2 4 6 8 10

Logic: Increase the counter by 2 so only even values are visited.

6. Multiplication table

Question: Read a number and print its multiplication table from 1 to 10.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", n, i, n * i);
    }
    return 0;
}
```

Sample: Input: 4 Output begins: 4 x 1 = 4 ... 4 x 10 = 40

Logic: The loop counter represents the multiplier.

7. Print numbers except multiples of 3 (continue)

Question: Read N and print numbers from 1 to N, skipping multiples of 3 using continue.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        if (i % 3 == 0) {
            continue;
        }
        printf("%d ", i);
    }
    return 0;
}
```

Sample: Input: 8 Output: 1 2 4 5 7 8

Logic: continue skips the remaining statements in the current iteration.

8. Stop printing when the counter reaches K (break)

Question: Read N and K. Print numbers from 1 to N, but stop immediately before printing K.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, k;
    scanf("%d %d", &n, &k);

    for (int i = 1; i <= n; i++) {
        if (i == k) {
            break;
        }
        printf("%d ", i);
    }
    return 0;
}
```

Sample: Input: 10 5 Output: 1 2 3 4

Logic: break exits the nearest enclosing loop immediately.

9. Correct an accidental infinite loop

Question: The following loop never ends. Correct it so that it prints 1 to 5 exactly once each: while (i <= 5) { printf("%d", i); }

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int i = 1;

    while (i <= 5) {
        printf("%d ", i);
        i++;                 // required update
    }
    return 0;
}
```

Sample: Output: 1 2 3 4 5

Logic: Without i++, the value remains 1 and i <= 5 stays true forever.

10. Even or odd using goto

Question: Read an integer and use goto with labels to print whether it is even or odd.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    if (n % 2 == 0)
        goto even;
    else
        goto odd;

even:
    printf("Even");
    return 0;

odd:
    printf("Odd");
    return 0;
}
```

Sample: Input: 12 Output: Even

Logic: goto transfers control to a labelled statement in the same function.

Part B · Nested loops and pattern programming

11. Solid square star pattern

Question: Read N and print an N × N square of stars.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 3\nOutput:\n* * *\n* * *\n* * *

Logic: Outer loop controls rows; inner loop prints N stars per row.

12. Increasing right-angled star triangle

Question: Read N and print 1 star in row 1, 2 in row 2, and so on.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n*\n* *\n* * *\n* * * *

Logic: The inner loop runs i times in row i.

13. Inverted star triangle

Question: Read N and print a triangle whose number of stars decreases from N to 1.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n* * * *\n* * *\n* *\n*

Logic: The outer loop counts down; the inner loop prints i stars.

14. Right-aligned star triangle

Question: Read N and print a right-aligned triangle using spaces before the stars.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++)
            printf(" ");
        for (int j = 1; j <= i; j++)
            printf("*");
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n *\n **\n ***\n****

Logic: Each row first prints n-i spaces, then i stars.

15. Pyramid star pattern

Question: Read N and print a centred pyramid with 1, 3, 5, ... stars.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++)
            printf(" ");
        for (int j = 1; j <= 2 * i - 1; j++)
            printf("*");
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 3\nOutput:\n *\n ***\n*****

Logic: Spaces decrease by one; stars are calculated as 2*i-1.

16. Increasing number triangle

Question: Print the sequence 1; then 1 2; then 1 2 3, continuing up to N.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n1\n1 2\n1 2 3\n1 2 3 4

Logic: Print j rather than i so each row restarts at 1.

17. Inverted number triangle

Question: Print 1 to N in the first row, 1 to N-1 in the second, continuing to 1.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n1 2 3 4\n1 2 3\n1 2\n1

Logic: The inner-loop limit follows the decreasing outer-loop variable.

18. Alphabet triangle

Question: Print A; then A B; then A B C, continuing up to N letters.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        char ch = 'A';
        for (int j = 1; j <= i; j++) {
            printf("%c ", ch);
            ch++;
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\nA\nA B\nA B C\nA B C D

Logic: Reset ch to 'A' at the start of every row.

19. Floyd's triangle

Question: Read N and print N rows of consecutive integers in a triangle.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, value = 1;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", value);
            value++;
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n1\n2 3\n4 5 6\n7 8 9 10

Logic: Unlike the previous number triangle, value is not reset at each row.

20. Hollow square pattern

Question: Print an N × N square with stars only on the boundary.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n)
                printf("* ");
            else
                printf("  ");
        }
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 4\nOutput:\n* * * *\n* *\n* *\n* * * *

Logic: A position is on the boundary if it is in the first/last row or first/last column.

Part C · Problem-solving using iteration

21. Factorial of a number

Question: Read N and calculate N!. Treat 0! as 1 and reject negative input.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    unsigned long long fact = 1;
    scanf("%d", &n);

    if (n < 0) {
        printf("Factorial is not defined");
    } else {
        for (int i = 1; i <= n; i++)
            fact *= i;
        printf("%llu", fact);
    }
    return 0;
}
```

Sample: Input: 5 Output: 120

Logic: Start fact at 1 because factorial is multiplication. For 0, the loop does not run and the answer stays 1.

22. Fibonacci series

Question: Read N and print the first N terms of the Fibonacci series starting with 0 and 1.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    long long a = 0, b = 1, next;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        printf("%lld ", a);
        next = a + b;
        a = b;
        b = next;
    }
    return 0;
}
```

Sample: Input: 7 Output: 0 1 1 2 3 5 8

Logic: Save the sum before shifting a and b to the next pair.

23. Reverse a number

Question: Read a non-negative integer and print its digits in reverse order.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, digit, rev = 0;
    scanf("%d", &n);

    while (n > 0) {
        digit = n % 10;
        rev = rev * 10 + digit;
        n = n / 10;
    }
    printf("%d", rev);
    return 0;
}
```

Sample: Input: 4821 Output: 1284

Logic: % 10 extracts the last digit; / 10 removes it. A reversed number may lose leading zeroes.

24. Sum of digits

Question: Read a non-negative integer and calculate the sum of its digits.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, sum = 0;
    scanf("%d", &n);

    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    printf("%d", sum);
    return 0;
}
```

Sample: Input: 583 Output: 16

Logic: The digits are 3, 8 and 5; their sum is 16. For input 0, the result remains 0.

25. Count digits in a number

Question: Read a non-negative integer and count how many digits it contains. Zero has one digit.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, count = 0;
    scanf("%d", &n);

    if (n == 0) {
        count = 1;
    } else {
        while (n > 0) {
            count++;
            n /= 10;
        }
    }
    printf("%d", count);
    return 0;
}
```

Sample: Input: 70025 Output: 5

Logic: Handle zero separately; otherwise the while loop would execute zero times.

26. Palindrome number

Question: Read a non-negative integer and check whether it reads the same forwards and backwards.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, original, digit, rev = 0;
    scanf("%d", &n);
    original = n;

    while (n > 0) {
        digit = n % 10;
        rev = rev * 10 + digit;
        n /= 10;
    }

    if (original == rev)
        printf("Palindrome");
    else
        printf("Not palindrome");

    return 0;
}
```

Sample: Input: 1221 Output: Palindrome

Logic: Keep the original number because n becomes 0 during digit extraction.

27. Prime number check

Question: Read N and determine whether it is prime. Numbers less than or equal to 1 are not prime.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, prime = 1;
    scanf("%d", &n);

    if (n <= 1) {
        prime = 0;
    } else {
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                prime = 0;
                break;
            }
        }
    }

    if (prime)
        printf("Prime");
    else
        printf("Not prime");

    return 0;
}
```

Sample: Input: 29 Output: Prime

Logic: A divisor is enough to prove a number is not prime. Checking up to sqrt(n) is sufficient.

28. Print all prime numbers from 1 to N

Question: Read N and print every prime number between 1 and N.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int num = 2; num <= n; num++) {
        int prime = 1;

        for (int d = 2; d * d <= num; d++) {
            if (num % d == 0) {
                prime = 0;
                break;
            }
        }

        if (prime)
            printf("%d ", num);
    }
    return 0;
}
```

Sample: Input: 10 Output: 2 3 5 7

Logic: For each candidate, reset prime to 1 before checking its divisors.

29. Armstrong number (three-digit version)

Question: Read a three-digit number and check whether the sum of the cubes of its digits equals the number.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, original, digit, sum = 0;
    scanf("%d", &n);
    original = n;

    while (n > 0) {
        digit = n % 10;
        sum += digit * digit * digit;
        n /= 10;
    }

    if (sum == original)
        printf("Armstrong");
    else
        printf("Not Armstrong");

    return 0;
}
```

Sample: Input: 153 Output: Armstrong

Logic: 153 = 1^3 + 5^3 + 3^3. This version is specifically for three-digit numbers.

30. Find the largest digit in a number

Question: Read a non-negative integer and print its largest digit.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, digit, largest = 0;
    scanf("%d", &n);

    while (n > 0) {
        digit = n % 10;
        if (digit > largest)
            largest = digit;
        n /= 10;
    }

    printf("%d", largest);
    return 0;
}
```

Sample: Input: 58329 Output: 9

Logic: Extract each digit and update largest whenever a bigger digit is found.

Part D · Mixed exam-style problems and quick revision

31. Count and sum even digits

Question: Read a non-negative integer. Print the sum of its even digits and how many even digits it has.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n, digit, sum = 0, count = 0;
    scanf("%d", &n);

    while (n > 0) {
        digit = n % 10;
        if (digit % 2 == 0) {
            sum += digit;
            count++;
        }
        n /= 10;
    }

    printf("Sum = %d\nCount = %d", sum, count);
    return 0;
}
```

Sample: Input: 48251 Output: Sum = 14, Count = 3

Logic: Use one loop to extract digits and an if condition to filter even digits.

32. Print a diamond pattern

Question: Read N and print a filled diamond with N rows in the upper half and N-1 rows in the lower half.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }

    for (int i = n - 1; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++) printf(" ");
        for (int j = 1; j <= 2 * i - 1; j++) printf("*");
        printf("\n");
    }
    return 0;
}
```

Sample: Input: 3\nOutput:\n *\n ***\n*****\n ***\n *

Logic: Build the upper pyramid, then mirror it downward without repeating the widest row.

33. Generate a menu-driven repeat program using do-while

Question: Repeatedly read a choice: 1 prints 1 to 5, 2 prints 5 to 1, and 0 exits. Use do-while.

Complete C solution:

```
#include <stdio.h>
int main(void) {
    int choice;

    do {
        scanf("%d", &choice);

        if (choice == 1) {
            for (int i = 1; i <= 5; i++) printf("%d ", i);
            printf("\n");
        } else if (choice == 2) {
            for (int i = 5; i >= 1; i--) printf("%d ", i);
            printf("\n");
        } else if (choice != 0) {
            printf("Invalid choice\n");
        }
    } while (choice != 0);

    return 0;
}
```

Sample: Input choices: 1, 2, 0 Output: 1 2 3 4 5 then 5 4 3 2 1

Logic: The do-while repeats until the exit choice 0 is entered.

Common errors that cost marks

• Forgetting i++ / i-- in a while loop can create an infinite loop.

• Using i < n instead of i <= n can omit the final value.

• Placing a semicolon directly after a for or while header creates an unintended empty loop.

• A do-while statement must end with a semicolon: while (condition);

• Reset the inner-loop counter for every new outer-loop iteration.

• Print the newline after the inner loop, not inside it, when each row should appear on a separate line.

• break exits the nearest loop; continue skips only the current iteration.

• Save the original number before reversing it for a palindrome check.

• Handle n = 0 explicitly in digit-count questions.

• For prime checking, 1 and all smaller integers are not prime.

Last-minute practice checklist

■ I can write the same counting task using for, while and do-while.

■ I can trace nested loops and calculate how many times the inner body executes.

■ I can use break and continue correctly.

■ I can identify and fix a loop that never terminates.

■ I can create a square, triangle, inverted triangle, right-aligned triangle and pyramid.

■ I can create number and alphabet patterns.

■ I can solve factorial, Fibonacci, reverse, digit sum, digit count and palindrome problems.

■ I can check primes and explain why 1 is not prime.

Source alignment: CAT-3 SPC notes for Lectures 16–20 and the syllabus map retrieved from the previous study session. Armstrong number, largest digit, even-digit aggregation, diamond and menu-driven repetition are included as useful variations for practising the announced iteration and nested-loop skills.